

인공지능의 이해

Understanding Artificial Intelligence

유준수 (Joon Soo Yoo, Ph.D.)

Fall 2026

Defense & Advanced Security Lab (DAS Lab)

Department of Advanced Defense Technology and Industry

Jeonbuk National University

Classification and Model Evaluation

From Classification to Model Evaluation (2)

Week 5

8. Recall

이번에는 실제 Gentoo Penguin만 생각해 보자.

Actual Gentoo
↓
20 Penguins
17 Predicted Gentoo
3 Predicted Adelie

질문은 다음과 같다.

“실제 Gentoo 중
얼마나 많은 Gentoo를 찾아냈는가?”

이를 Recall이라고 한다.

핵심

Recall은 실제 Positive 중 Model이 Positive라고 찾아낸 비율이다.

8-1. (참고) Recall Formula

Recall은 다음과 같이 계산한다.

$$Recall = \frac{TP}{TP + FN}$$

앞의 Example에서는

$$TP = 17, \quad FN = 3$$

이므로,

$$Recall = \frac{17}{17 + 3} = \frac{17}{20} = 0.85$$

이다.

즉 실제 Gentoo Penguin의 85%를 정확하게 찾아냈다.

참고

Recall은 실제 Positive를 얼마나 놓치지 않고 찾아냈는지를 나타낸다.

8-2. Precision vs. Recall

Precision과 Recall은 서로 다른 질문에 답한다.

Metric	Question
Precision	Gentoo라고 Prediction한 것 중 실제 Gentoo는 얼마나 되는가?
Recall	실제 Gentoo 중 Model이 얼마나 많이 찾아냈는가?

핵심

Precision과 Recall은 Classification Model의 서로 다른 특성을 평가한다.

9. F1 Score

Precision과 Recall을 하나의 값으로 함께 보고 싶을 수 있다.

이를 위해 F1 Score를 사용할 수 있다.

예를 들어,

Precision = 0.90

Recall = 0.85

이라면 두 Metric 모두 비교적 높은 값을 가진다.

F1 Score는 Precision과 Recall을 함께 고려하는 Metric이다.

핵심

F1 Score는 Precision과 Recall 사이의 균형을 하나의 숫자로 표현한다.

9-1. (참고) F1 Score Formula

F1 Score는 Precision과 Recall의 Harmonic Mean을 사용한다.

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

예를 들어,

$$Precision = 0.90, \quad Recall = 0.85$$

이면,

$$F1 = 2 \frac{0.90 \times 0.85}{0.90 + 0.85} \approx 0.874$$

이다.

참고

Precision과 Recall 중 하나가 매우 낮으면 F1 Score도 낮아지는 경향이 있다.

9-2. Which Metric Should We Use?

Classification Model은 여러 관점에서 평가할 수 있다.

Metric	Meaning
Accuracy	전체 Prediction 중 얼마나 많이 맞혔는가?
Precision	Positive라고 Prediction한 것 중 얼마나 맞았는가?
Recall	실제 Positive 중 얼마나 많이 찾아냈는가?
F1	Precision과 Recall을 함께 고려하면 어떠한가?

어떤 Metric이 중요한지는 Problem의 목적에 따라 달라질 수 있다.

핵심

하나의 Metric만 보고 Model을 판단하기보다 Problem의 목적에 맞는 Metric을 선택해야 한다.

10. Classification Threshold

Logistic Regression은 Probability를 출력할 수 있다.

예를 들어,

$$P(\text{Gentoo}) = 0.73$$

Threshold가 0.5라면,

$$0.73 \geq 0.5$$

이므로 Gentoo라고 Classification한다.

하지만 Threshold를 0.8로 바꾸면,

$$0.73 < 0.8$$

이므로 Adelie라고 Classification한다.

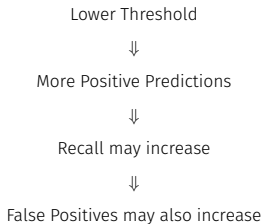
핵심

같은 Model과 같은 Probability라도 Threshold에 따라 최종 Classification 결과가 달라질 수 있다.

10-1. What Happens When the Threshold Changes?

Gentoo를 Positive라고 생각해보자.

Threshold를 낮추면 Gentoo라고 판단하기 쉬워진다.



반대로 Threshold를 높이면 Gentoo라고 판단하기 어려워진다.

핵심

Threshold를 바꾸면 TP, FP, FN, TN이 변하고, 따라서 Precision과 Recall도 달라질 수 있다.

10-2. Example: Lowering the Threshold

Model이 다섯 Penguin에 대해 다음 Gentoo Probability를 출력했다고 하자.

Penguin	P(Gentoo)	Actual
A	0.90	Gentoo
B	0.75	Gentoo
C	0.55	Gentoo
D	0.40	Gentoo
E	0.20	Adelie

Threshold = 0.50이면,

A, B, C → Gentoo

D, E → Adelie

실제 Gentoo 4마리 중 3마리를 찾았다.

$$Recall = \frac{3}{4} = 0.75$$

Threshold를 0.3으로 낮추면,

A, B, C, D → Gentoo

이제 실제 Gentoo 4마리를 모두 찾는다.

$$Recall = \frac{4}{4} = 1.0$$

핵심

Threshold를 낮추면 Positive라고 판단하기 쉬워져 실제 Positive를 더 많이 찾아낼 수 있고 Recall이 증가할 수 있다.

10-3. But What Can Go Wrong?

이번에는 Penguin D가 실제로 Adelie라고 생각해보자.

Penguin	P(Gentoo)	Actual
A	0.90	Gentoo
B	0.75	Gentoo
C	0.55	Gentoo
D	0.40	Adelie
E	0.20	Adelie

Threshold = 0.5이면,

$$0.40 < 0.5$$

이므로 D를 Adelie라고 올바르게 판단한다.

하지만 Threshold = 0.3으로 낮추면,

$$0.40 \geq 0.3$$

이므로 D를 Gentoo라고 잘못 판단한다.

Actual Adelie → Predicted Gentoo



False Positive

핵심

Threshold를 낮추면 Recall이 증가할 수 있지만, Negative까지 Positive라고 판단하여 False Positive가 증가하고 Precision이 낮아질 수도 있다.

10-4. Precision-Recall Trade-off

Threshold를 변경하면 Precision과 Recall 사이에 Trade-off가 나타날 수 있다.

Threshold	Precision	Recall
0.3	Lower	Higher
0.5	Balanced	Balanced
0.8	Higher	Lower

이 표는 일반적인 경향을 단순화하여 나타낸 것이다.

실제 값은 Data와 Model에 따라 달라진다.

핵심

Threshold는 고정된 절대적인 정답이 아니라, Problem의 목적에 따라 조정할 수 있다.

11. Does the Train-Test Split Matter?

같은 Dataset과 같은 Model을 사용해도 Train-Test Split이 달라질 수 있다.

예를 들어,

Split A

Train: Penguin 1, 2, 4, 5, ...

Test: Penguin 3, 8, ...

Split B

Train: Penguin 1, 3, 6, 8, ...

Test: Penguin 2, 4, ...

Training에 사용되는 Data가 달라지면 학습된 Model도 조금 달라질 수 있다.

핵심

같은 Algorithm을 사용하더라도 어떤 Data로 Training했는지에 따라 학습된 Model과 성능이 달라질 수 있다.

11-1. Random Train-Test Split

일반적으로 Data를 Random하게 나누어 Training Data와 Test Data를 만든다.

예를 들어 Python에서는

Source Code

```
train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

와 같이 사용할 수 있다.

- ▶ *test_size=0.2*: 20%를 Test Data로 사용
- ▶ *random_state*: Random Split을 재현하기 위한 값

핵심

Random Split을 사용하면 어떤 Sample이 Training 또는 Test에 포함되는지가 달라질 수 있다.

11-2. Different Split, Different Result

같은 Logistic Regression Model이라도 Data Split에 따라 평가 결과가 달라질 수 있다.

예를 들어,

Random State	Training Data	Accuracy
1	Split A	0.91
10	Split B	0.94
42	Split C	0.92

위 숫자는 개념 설명을 위한 Example이다.

중요한 질문은 다음과 같다.

Which result should we trust?

핵심

한 번의 Train-Test Split 결과만으로 Model의 일반적인 성능을 판단하면 불안정할 수 있다.

11-3. Do Not Choose the Lucky Split

여러 Random Split을 시도한 뒤 가장 높은 Accuracy만 선택하면 될까?

예를 들어,

Random State	Accuracy
1	91%
10	94%
42	92%
100	97%

97%가 나왔다는 이유만으로 `random_state=100`을 선택하는 것은 적절하지 않다.

특정 Test Data에서 우연히 좋은 결과가 나온 것일 수도 있기 때문이다.

중요

Test 결과를 보고 유리한 Data Split을 선택하면 Model 성능을 과대평가할 수 있다.

12. Cross Validation

한 번의 Train-Test Split에 너무 의존하지 않는 방법이 필요하다.

Cross Validation에서는 Data를 여러 부분으로 나누고, Training과 Validation을 반복한다.

Split 1 → Score 1

Split 2 → Score 2

Split 3 → Score 3

⋮

Multiple Scores

⇓

Average Performance

핵심

Cross Validation은 여러 Data Split에서 Model을 평가하여 성능을 보다 안정적으로 확인하는 방법이다.

12-1. K-Fold Cross Validation

대표적인 방법이 K-Fold Cross Validation이다.

예를 들어 5-Fold Cross Validation에서는 Data를 5개 Group으로 나눈다.

	F1	F2	F3	F4	F5
Round 1	Test	Train	Train	Train	Train
Round 2	Train	Test	Train	Train	Train
Round 3	Train	Train	Test	Train	Train
Round 4	Train	Train	Train	Test	Train
Round 5	Train	Train	Train	Train	Test

모든 Data가 한 번씩 Evaluation에 사용된다.

핵심

K-Fold Cross Validation은 Data를 K개로 나누고, 각 Fold를 번갈아 Evaluation에 사용한다.

12-2. (참고) Cross Validation Score

K-Fold Cross Validation에서는 K개의 Evaluation Score가 생성된다.

예를 들어 5-Fold 결과가 다음과 같다고 하자.

$$S_1, S_2, S_3, S_4, S_5$$

평균 성능은

$$\bar{S} = \frac{1}{5} \sum_{i=1}^5 S_i$$

로 계산할 수 있다.

예를 들어,

$$0.90, 0.94, 0.92, 0.91, 0.93$$

이라면,

$$\bar{S} = 0.92$$

이다.

참고

Cross Validation에서는 하나의 Score보다 여러 Fold의 평균 성능을 확인할 수 있다.

12-3. What Does Cross Validation Tell Us?

두 Model의 Cross Validation 결과를 생각해보자.

Model	F1	F2	F3	F4	F5
Model A	.92	.91	.93	.92	.92
Model B	.99	.82	.96	.84	.98

Model B는 어떤 Split에서는 매우 좋지만 다른 Split에서는 성능이 크게 떨어진다.

따라서 평균뿐 아니라 Score가 얼마나 안정적인지도 살펴볼 수 있다.

핵심

Cross Validation은 Model의 평균적인 성능뿐 아니라 Data Split에 따른 성능 변화도 확인하게 해준다.

13. From Threshold to ROC Curve

앞에서 Threshold에 따라 Classification 결과가 달라지는 것을 살펴보았다.

예를 들어 Threshold를

0.2

0.4

0.5

0.6

0.8

처럼 계속 바꾸면,

각 Threshold마다

- ▶ True Positive
- ▶ False Positive
- ▶ True Negative
- ▶ False Negative

가 달라진다.

핵심

ROC Curve는 하나의 Threshold만 보는 것이 아니라, 여러 Threshold에서 Model의 Classification 특성을 평가한다.

13-1. (참고) True Positive Rate

ROC Curve에서는 True Positive Rate를 사용한다.

True Positive Rate는

$$TPR = \frac{TP}{TP + FN}$$

이다.

그런데 이것은 앞에서 배운 Recall과 같다.

$$TPR = Recall$$

즉 실제 Positive 중 얼마나 많은 Positive를 찾아냈는지를 나타낸다.

참고

True Positive Rate는 Recall과 같은 Metric이다. ROC Curve의 y-axis에 사용된다.

13-2. (참고) False Positive Rate

ROC Curve의 다른 Metric은 False Positive Rate이다.

$$FPR = \frac{FP}{FP + TN}$$

이는 실제 Negative 중 얼마나 많은 Sample을 Positive라고 잘못 판단했는지를 나타낸다.

예를 들어 실제 Adelie가 20마리인데 2마리를 Gentoo라고 잘못 Classification했다면,

$$FPR = \frac{2}{20} = 0.1$$

이다.

참고

False Positive Rate는 실제 Negative를 얼마나 자주 Positive라고 잘못 Classification했는지 나타낸다.

13-3. (참고) ROC Curve

ROC Curve는 여러 Threshold에서 TPR과 FPR의 관계를 나타낸다.

y-axis

True Positive Rate (TPR)

vs.

x-axis

False Positive Rate (FPR)

각 Point는 서로 다른 Threshold에서의 Classification 결과를 나타낸다.

참고

ROC Curve는 Threshold를 변화시키면서 Positive를 잘 찾는 정도와 False Positive의 관계를 보여준다.

13-4. (참고) Understanding the ROC Curve

좋은 Classifier는 가능한 한

- ▶ True Positive Rate는 높고
- ▶ False Positive Rate는 낮기를 원한다.

따라서 ROC Curve에서는 왼쪽 위에 가까운 성능이 좋은 특성을 나타낸다.

Low FPR
+
High TPR
↓
Better Classification

Random Classification은 대략 대각선에 가까운 특성을 보인다.

참고

ROC Curve는 특정 Threshold 하나가 아니라 Model의 전반적인 Class Separation 성능을 살펴보는 방법이다.

14. (참고) What is AUC?

AUC는 Area Under the ROC Curve의 약자이다.

즉 ROC Curve 아래의 면적을 하나의 숫자로 표현한다.

AUC	Interpretation
Close to 1.0	Strong Class Separation
Around 0.5	Similar to Random Ranking

AUC가 높을수록 일반적으로 Positive와 Negative를 잘 구분하는 Ranking을 만든다.

참고

AUC는 여러 Threshold에서 나타나는 ROC 성능을 하나의 숫자로 요약한 Metric이다.

14-1. (참고) Accuracy vs. AUC

Accuracy와 AUC는 서로 다른 것을 평가한다.

Metric	Main Idea
Accuracy	정해진 Classification 결과를 얼마나 많이 맞혔는가?
AUC	여러 Threshold를 고려할 때 Positive와 Negative를 얼마나 잘 구분하는가?

Accuracy는 특정 Classification Decision에 의존한다.

AUC는 Threshold를 변화시키면서 Model의 Ranking / Separation 성능을 본다.

참고

Metric마다 Model의 서로 다른 특성을 평가하므로, 하나의 Metric만으로 모든 성능을 설명할 수는 없다.

15. Comparing the Three Models

이제 처음에 만든 세 Model로 돌아가보자.

Model	Features
A	Bill Length
B	Bill Length + Flipper Length
C	Bill Length + Flipper Length + Body Mass

각 Model에 대해 다음 Metric을 비교할 수 있다.

- ▶ Accuracy
- ▶ Precision
- ▶ Recall
- ▶ F1 Score
- ▶ Cross Validation Score
- ▶ AUC

핵심

같은 Logistic Regression Algorithm을 사용하더라도 사용하는 Feature에 따라 서로 다른 Model이 학습될 수 있으며, Evaluation Metric을 이용하여 그 특성을 비교할 수 있다.

15-1. Example: Model Comparison

세 Model의 평가 결과가 다음과 같다고 생각해 보자.

Model	Accuracy	Precision	Recall	F1
A	0.88	0.90	0.84	0.87
B	0.93	0.92	0.94	0.93
C	0.94	0.93	0.95	0.94

위 숫자는 개념 설명을 위한 Example이다.

Model B와 C의 성능은 비슷하지만, Model C가 Feature를 하나 더 사용한다.

따라서 단순히

“Feature가 많을수록 좋다”

라고 결론 내릴 수는 없다.

핵심

Model을 비교할 때는 Prediction Performance뿐 아니라 어떤 Information을 사용하는지도 함께 살펴볼 수 있다.

15-2. More Features vs. Better Performance

Feature를 추가했을 때 Model의 성능이 얼마나 개선되었는지 생각해보자.

Model	# Features	Accuracy
A	1	0.88
B	2	0.93
C	3	0.94

Model A에서 B로 갈 때는

$0.88 \rightarrow 0.93$

으로 비교적 큰 변화가 있었다.

하지만 Model B에서 C로 갈 때는

$0.93 \rightarrow 0.94$

로 변화가 작다.

질문

추가적인 Feature 하나가 얼마나 유용한 Information을 제공했는가?

15-3. Model Evaluation Is More Than One Number

Model의 성능을 하나의 숫자로만 판단하면 중요한 특성을 놓칠 수 있다.

예를 들어 두 Model이 있다고 하자.

Model	Accuracy	Precision	Recall
A	0.92	0.97	0.80
B	0.92	0.85	0.96

Accuracy는 동일하지만 Precision과 Recall은 다르다.

즉 두 Model은 서로 다른 Classification 특성을 가진다.

핵심

같은 Accuracy를 가진 Model이라도 Error의 종류와 Precision/Recall 특성은 다를 수 있다.

15-4. Which Model Should We Use?

어떤 Model을 사용할지는 Problem의 목적에 따라 달라질 수 있다.

예를 들어,

Goal	Important Question
Overall Classification	전체적으로 얼마나 잘 맞히는가?
Reliable Positive Prediction	Positive라고 한 Prediction을 얼마나 믿을 수 있는가?
Find More Positives	실제 Positive를 얼마나 놓치지 않는가?
Balanced Performance	Precision과 Recall이 모두 괜찮은가?

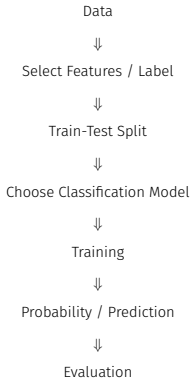
따라서 Model Selection에서는 Problem의 목적과 Evaluation Metric을 함께 고려한다.

핵심

“가장 좋은 Model”은 하나의 Metric만으로 결정되는 것이 아니라, 해결하려는 Problem과 평가 기준에 따라 달라질 수 있다.

16. The Classification Workflow

이번 Week에서 배운 전체 과정을 연결해보자.



핵심

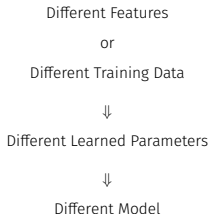
Classification도 Week 2에서 배운 Machine Learning Workflow를 그대로 따른다.

16-1. What Can Change the Learned Model?

같은 Logistic Regression을 사용하더라도 여러 요소에 의해 학습 결과가 달라질 수 있다.

- ▶ 어떤 Feature를 사용하는가?
- ▶ 어떤 Training Data를 사용하는가?
- ▶ Data의 양과 품질은 어떠한가?
- ▶ Model의 Hyperparameter는 어떻게 설정했는가?

예를 들어,



핵심

Algorithm의 이름이 같다고 해서 항상 동일한 학습 결과가 만들어지는 것은 아니다.

16-2. Parameters vs. Hyperparameters

Machine Learning에서는 Parameter와 Hyperparameter를 구분한다.

Parameter

Training 과정에서 Data로부터 학습된다.

Logistic Regression의 예:

$$w_1, w_2, \dots, w_d, b$$

Hyperparameter

Training 전에 사람이 설정한다.

예:

- ▶ Regularization 설정
- ▶ Solver 설정
- ▶ Maximum Iterations

핵심

Parameter는 Model이 Data에서 학습하고, Hyperparameter는 Training 방법을 조절하기 위해 설정한다.

16-3. (참고) Logistic Regression Parameters

Logistic Regression Model을 다시 생각해보자.

$$P(y = 1|x) = \sigma(w^T x + b)$$

Feature가 하나라면,

$$w_1 x_1 + b$$

를 학습한다.

Feature가 세 개라면,

$$w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

를 학습한다.

즉 Feature가 달라지면 학습해야 하는 Parameter의 구조도 달라진다.

참고

Logistic Regression Training은 주어진 Feature와 Training Data를 이용하여 적절한 Weight와 Bias를 찾는 과정이다.

17. What We Learned

이번 Week에서는 Classification의 전체 과정을 살펴보았다.

- ▶ Classification은 Category를 Prediction한다.
- ▶ Logistic Regression을 Classification에 사용할 수 있다.
- ▶ Model은 Probability를 계산하고 Threshold로 Class를 결정할 수 있다.
- ▶ Feature가 달라지면 학습된 Model도 달라질 수 있다.
- ▶ Accuracy는 전체 Prediction의 정답 비율이다.
- ▶ Confusion Matrix는 Error의 종류를 보여준다.
- ▶ Precision과 Recall은 서로 다른 관점에서 Model을 평가한다.
- ▶ F1 Score는 Precision과 Recall을 함께 고려한다.
- ▶ Threshold에 따라 Classification 결과가 달라질 수 있다.
- ▶ Cross Validation은 여러 Data Split에서 Model을 평가한다.
- ▶ ROC Curve와 AUC는 여러 Threshold에서의 성능을 평가할 수 있다.

17-1. Five Questions to Remember

Classification Problem을 보면 다음 질문을 생각해보자.

1. **What do we want to classify?**
어떤 Class를 Prediction하려는가?
2. **What Features do we use?**
어떤 Information으로 판단할 것인가?
3. **What Model do we use?**
어떤 Classification Algorithm을 사용할 것인가?
4. **How do we evaluate it?**
Accuracy, Precision, Recall, F1 등의 어떤 Metric을 사용할 것인가?
5. **Does it work on new Data?**
Test Data 또는 Cross Validation에서도 잘 동작하는가?

17-2. Regression and Classification

Week 2와 Week 3를 함께 정리해보자.

	Regression	Classification
Goal	Number	Class
Example	Exam Score	Penguin Species
Example Model	Linear Regression	Logistic Regression
Prediction	82.4	Gentoo
Evaluation	MAE	Accuracy / F1 / AUC

하지만 기본 Workflow는 동일하다.

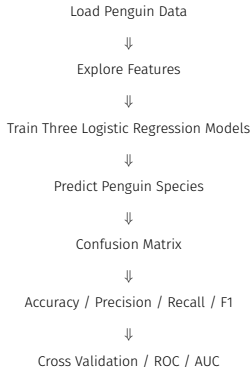
Data → Training → Model → Prediction → Evaluation

핵심

문제와 Model은 달라져도 Machine Learning의 기본적인 사고방식은 반복된다.

17-3. Now, Let's Classify Penguins

이제 Google Colab에서 실제 Penguin Data를 Classification한다.



Next

같은 Dataset에 서로 다른 Feature를 사용하여 세 개의 Classification Model을 만들고 직접 비교한다.

Thank You

Questions?